



PDF Download
3731569.3764803.pdf
29 March 2026
Total Citations: 2
Total Downloads: 4251



Published: 12 October 2025

Citation in BibTeX format

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764803>

RESEARCH-ARTICLE

Unlocking True Elasticity for the Cloud-Native Era with Dandelion

TOM KUCHLER, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

PINGHE LI, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

YAZHUO ZHANG, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

LAZAR CVETKOVIĆ, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

BORIS GORANOV, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

TOBIAS STOCKER, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

[View all](#)

Open Access Support provided by:

Swiss Federal Institute of Technology, Zurich

Max Planck Institute for Software Systems



Unlocking True Elasticity for the Cloud-Native Era with Dandelion

Tom Kuchler
ETH Zurich
Switzerland

Pinghe Li
ETH Zurich
Switzerland

Yazhuo Zhang
ETH Zurich
Switzerland

Lazar Cvetković
ETH Zurich
Switzerland

Boris Goranov
ETH Zurich
Switzerland

Tobias Stocker
ETH Zurich
Switzerland

Leon Thomm
ETH Zurich
Switzerland

Simone Kalbermatter
ETH Zurich
Switzerland

Tim Notter
ETH Zurich
Switzerland

Andrea Lattuada
MPI-SWS
Germany

Ana Klimovic
ETH Zurich
Switzerland

Abstract

Elasticity is fundamental to cloud computing. An elastic platform can quickly allocate resources to match the demand of each workload as it arrives, rather than pre-provisioning resources to meet performance objectives. However, even serverless platforms — which boot sandboxes in 10s to 100s of milliseconds — are not sufficiently elastic to avoid pre-provisioning expensive resources. Today’s FaaS platforms provision many extra, idle sandboxes in memory to reduce the occurrence of slow, cold starts. Initializing securely isolated sandboxes with a POSIX-like computing environment that today’s cloud users expect is slow as it requires booting a guest OS and configuring networking.

Our key insight is that the rise of cloud-native application development provides an opportunity to rethink the application interface to the cloud and co-design a much more efficient, elastic computing platform under the hood. We propose Dandelion, an elastic cloud platform with a declarative cloud-native programming model that replaces POSIX-based network interfaces with higher-level (e.g., HTTP-based) interfaces for applications to interact with remote services like cloud storage, databases, and AI inference services. Dandelion executes applications expressed as DAGs of pure compute functions and communication functions.

This enables Dandelion to securely execute compute functions in lightweight sandboxes that cold start in *100s of microseconds*, since pure functions do not rely on software environments such as a guest OS. Dandelion makes it practical to boot sandboxes on-demand per request, decreasing performance variability by two to three orders of magnitude

compared to Firecracker and reducing committed memory by 96% on average when running the Azure Functions trace.

CCS Concepts: • Computer systems organization → Cloud computing.

ACM Reference Format:

Tom Kuchler, Pinghe Li, Yazhuo Zhang, Lazar Cvetković, Boris Goranov, Tobias Stocker, Leon Thomm, Simone Kalbermatter, Tim Notter, Andrea Lattuada, and Ana Klimovic. 2025. Unlocking True Elasticity for the Cloud-Native Era with Dandelion. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3731569.3764803>

1 Introduction

Elasticity is the degree to which a system is able to adapt to workload changes by provisioning and deprovisioning resources in an autonomic manner, such that at each point in time, the available resources match the current demand as closely as possible [52].

Elasticity is a key selling point of cloud computing [21]. The more elastic the infrastructure, the more cost-efficient cloud computing can become, since fewer resources need to be pre-provisioned before a request arrives to ensure low latency. Infrastructure as a Service (IaaS) offers greater elasticity than purchasing on-premise servers, as users can boot virtual machines (VMs) in tens of seconds [111] and rent them for as long as they need. Serverless computing [29] pushes elasticity even further, as Function as a Service (FaaS) platforms boot tasks in tens to hundreds of milliseconds [16, 42, 104] and autoscale sandboxes based on invocation load [95].

However, even serverless cloud platforms are not elastic enough to avoid pre-provisioning expensive resources. To achieve low latency and high throughput, FaaS platforms keep many idle function sandboxes ready in memory [16, 95]. For example, when running the Azure Functions trace [95], the Knative [7] serverless platform aggressively autoscales sandboxes such that 97% of requests execute in sandboxes



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, October 13–16, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10.

<https://doi.org/10.1145/3731569.3764803>

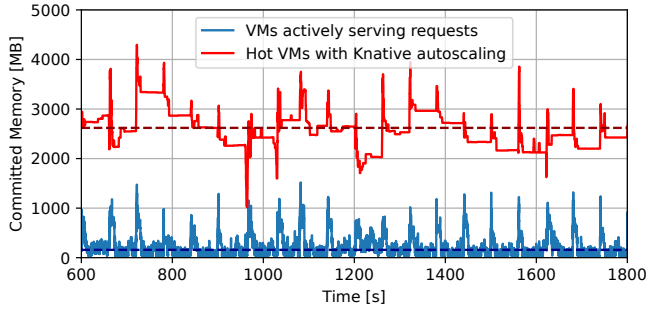


Figure 1. Azure trace execution with 100 functions, comparing committed memory for all VMs on the node with Knative autoscaling vs. for the VMs actively serving requests. Keeping VMs hot reduces cold starts, but consumes extra DRAM, which is expensive. Dotted lines show the average.

whose memory was provisioned before the invocation arrived (warm starts). Although this improves median latency, it causes the platform to commit 16× more memory on average (red line in Figure 1) than the memory required for running active requests (blue line). Provisioning so many extra sandboxes in memory is expensive, since DRAM dominates server costs [67, 113].

The current approach of capping costs by pre-provisioning sandboxes for many, but not all functions leads to the additional problem of high performance variability. The small percentage of requests that are cold starts incur noticeable extra latency to boot a sandbox on the critical path. Figure 2 shows how sensitive tail latency is to the percentage of hot requests (note the log scale!), despite sandbox creation optimizations like snapshots [5]. The high tail latency due to cold starts particularly impacts user-facing tasks like web serving and interactive data processing.

We show that a key barrier to achieving true elasticity in the cloud is executing each function in a sandbox derived from a container or VM that *exposes a POSIX-like interface to the function*. A minimal KVM [85] instance can boot in less than 140 thousand cycles [109] (i.e. $\sim 52\mu s$ at 2.69GHz). However, booting a hello-world function in a Firecracker MicroVM (which is based on KVM) takes over 10ms even with snapshots. Much of this time is spent on loading the guest OS snapshot and configuring the MicroVM’s network setup ($>8ms$ is spent on demand paging the snapshot and re-establishing the guest-host connection), which are needed to provide a POSIX-like interface to the user function.

Although prioritizing POSIX compatibility was essential in the early days of cloud computing to ease migration from on-premise deployments, cloud-native application development is increasingly common today. Developers build applications for cloud environments, leveraging numerous cloud services for storage, analytics, AI inference, and more, which are all exposed over REST APIs [23]. Cloud-native programming is evolving [18, 33, 41, 43, 54, 63, 64, 71, 83, 98, 102, 111, 117], with systems like SigmaOS [101] demonstrating

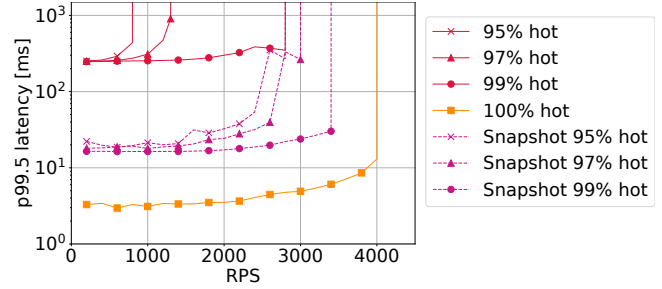


Figure 2. 128x128 int64 matmul running in Firecracker MicroVMs. The % of cold requests greatly impacts performance.

that well-tailored interfaces for cloud-native applications can eliminate the need for traditional POSIX-like interfaces. Even today’s FaaS platforms, such as AWS Lambda, are not fully POSIX compliant, as functions cannot accept incoming connections [112]. Hence, now is the right time to rethink cloud abstractions that can enable the underlying infrastructure to become more efficient and truly elastic.

We propose Dandelion, an elastic cloud platform with a declarative cloud-native programming model that replaces the POSIX-based network interface with a higher-level (e.g., HTTP-based) interface for applications to interact with remote services like cloud storage, databases, and AI inference services. Dandelion executes applications expressed as direct acyclic graphs (DAGs) of *pure compute functions* and *communication functions*. Pure compute functions are bodies of (untrusted) user code, which take a declared list of inputs and produce a declared list of outputs. Communication functions are implemented by the Dandelion platform and can be invoked (but not modified) in user application DAGs. Dandelion currently implements a communication function for HTTP communication, which enables REST API interactions. We plan to add more communication functions to support additional protocols.

Decomposing a cloud application into pure compute and communication functions enables redesigning the function execution system for high elasticity. First, Dandelion can securely execute pure compute functions in lightweight sandboxes that are fast to cold start, since the sandboxes do not need a guest OS setup for networking, threading, etc. or an isolated virtual network device on the host. Second, Dandelion can dedicate a CPU core for each compute function and run it to completion to minimize context switches, since a pure function does not block.¹ Finally, it enables Dandelion to run a cooperative runtime for network I/O per node (rather than per sandbox) and elastically adjust the CPU core allocation based on the number of compute vs. communication functions in the system.

¹Compute functions do not issue syscalls. Dandelion allocates an isolated memory region for each function before it starts executing, loads inputs in this region (e.g., via a memory-mapped file system), and provides a standard C/C++ library that allows file read/writes without syscalls (see §4.1).

We implement several applications on Dandelion, including a distributed log processing application (which interacts with an authentication service and storage servers over HTTP), SQL query processing (which ingests data from S3), and a simple Text2SQL agentic AI workflow (which uses communication functions to prompt an LLM and to query a remote database). The platform design is not tied to a particular sandbox mechanism. To demonstrate this, we implement Dandelion with four different lightweight isolation backends for compute functions (KVM [85], Linux processes, CHERI memory capabilities [110], and rWasm [27]). We compare Dandelion’s performance and security trade-offs to current FaaS platforms that use Firecracker [16], gVisor [6], or Wasmtime [17] for isolation. Dandelion boots sandboxes in 100s of μ s, or even under 90μ s for CHERI-based sandboxes, which is over an order of magnitude faster than Firecracker snapshot booting. Dandelion makes it practical to cold-start every request, reducing memory overprovisioning by 96% compared to Knative autoscaling when replaying the Azure Functions trace and reducing execution time variance. Dandelion improves elasticity for query processing compared to the AWS Athena serverless query service, achieving 40% lower latency and 67% lower cost for short-running queries. The code for Dandelion is available at <https://github.com/eth-easl/dandelion>.

2 Serverless As We Know It Today

FaaS platforms are advertised as elastic cloud services that quickly provision and autoscale server infrastructure for users based on application load. We review their current execution system design, programming model, and discuss how current platforms provide the *illusion* of high elasticity.

2.1 FaaS Execution Environment

FaaS platforms execute and isolate functions by wrapping function code, inputs, outputs, and intermediate execution state inside sandboxes, such as MicroVMs [16], hardened containers [6], or Wasm modules [9]. Sandboxes are not shared across users or functions [94]. Subsequent invocations of the same function from the same user may reuse a sandbox [19]. The platform scales the number of sandboxes per function based on invocations [7, 95].

Hyperscaler cloud providers use hardware virtualization to sandbox untrusted user code [16, 20, 78]. Each function runs in a MicroVM with a guest OS. To breach security, a malicious user function would need to exploit both the guest OS and the hypervisor. The defense-in-depth provided by the VM is critical when user functions can issue a variety of system calls, as the syscall interface is considered a large attack surface prone to vulnerabilities [31, 51].

Traditional VMs can take tens of seconds to boot [111]. Firecracker MicroVMs [16] reduce boot times to hundreds of milliseconds by removing many general-purpose VMM

features that FaaS does not need, such as a BIOS and various device emulators. To further reduce boot time to tens of milliseconds [42, 104], MicroVMs can be restored from snapshots. Unikernel-based VMs [28, 65, 74, 76] reduce boot time to the millisecond range by creating a specialized, single-address-space machine image of the user function with only its necessary kernel functionality.

Some cloud or edge platforms run functions as Wasm modules [4, 44, 49, 77, 97, 107]. Wasm is a binary instruction format designed for a stack-based VM to run code at near-native speed with memory safety guarantees provided by a compiler and runtime [9]. Wasm sandboxes boot much faster than MicroVMs, which makes it practical to create sandboxes on-demand per request. However, Wasm code does not necessarily run as fast as native code [49, 115, 116]. Current Wasm compilers may lack optimizations implemented in more mature compilers like Clang, which can result in more instructions, CPU register pressure, and extra branches [55], though this is an active area of research and engineering. Our experiments in §7.3 and §7.6 show that Wasmtime [17] runs slower than native code for compute-intensive tasks.

2.2 FaaS Programming Model

FaaS developers submit code snippets that have a POSIX-like environment and can initiate communication with cloud services. However, they do not expose the public IP addresses of functions and functions cannot accept network connections [112]. Hence, functions that need to communicate typically exchange data via remote storage [61, 82]. Most FaaS platforms orchestrate functions as independent, stateless units. Some frameworks allow users to express dependencies and build workflows [11, 24, 92]. However, data dependencies are primarily used as a hint to spin up sandboxes for downstream functions, rather than to optimize data transfers between them. These workflows do not necessarily capture an application’s entire dataflow, as an application can still communicate with cloud services within a function [41, 71, 117].

2.3 Why FaaS is not Elastic Today

Although MicroVM snapshots reduce sandbox boot times down to tens of milliseconds, this latency is still too high to incur for every function invocation. Production traces reveal that many FaaS functions execute for tens of milliseconds or less [40, 59] and would hence experience high overhead. It is difficult to further reduce boot latency, as we find that at least 8ms are spent on loading a minimal snapshot (primarily the guest OS state) by demand paging and on re-establishing the network connection between the host and the guest process (to provide function inputs and retrieve its outputs). Both are necessary in a POSIX-like sandbox environment.

Using Wasm sandboxes is an option to reduce boot times compared to MicroVMs. However, in practice several commercial Wasm-based serverless platforms still apply defense-in-depth strategies for stronger isolation even though some

components of the Wasm toolchain have been formally verified [27, 57, 58]. For example, Cloudflare runs multiple instances of their Wasm runtime as separate processes on each machine to separate low-trusted Wasm workers from more trusted ones. They also use Linux namespaces and seccomp for additional sandboxing at the process level [36]. This limits elasticity, as scaling now involves starting a Wasm runtime (i.e. loading and initialization overheads) as well as setting up a namespace and seccomp filters.

Since quickly initializing a securely isolated POSIX-like environment for cloud user functions is difficult, today's platforms pre-provision sandboxes to reduce the probability of incurring guest OS and networking initialization overheads on the critical path of requests. This gives the illusion of microsecond-scale elasticity at the expense of consuming extra memory.

Figure 1 compares the committed memory on a server executing a sample of the Azure Functions trace [95] with Knative's default autoscaling algorithm that provisions extra idle sandboxes (red line) versus the memory usage of active sandboxes over time (blue line). Running a guest OS inside each function sandbox further adds to the memory footprint [69, 90]. Memory usage contributes significantly to the overall cost of operating a FaaS platform, as DRAM dominates server costs [67, 113]. This ultimately leads to higher cost for users. Since it is usually prohibitively expensive for providers to keep enough sandboxes hot in memory for *all* requests [7, 16, 48, 95], some requests incur sandbox creation on the critical path, introducing high performance variability for user applications, as shown in Figure 2.

3 Dandelion's Approach

To achieve high elasticity while eliminating the need to pre-provision idle sandboxes in memory, we propose Dandelion. Our key insight is that redesigning the traditional POSIX-based programming interface for cloud applications in the cloud-native era enables co-designing a more efficient and elastic cloud platform. We observe that cloud-native applications typically consist of interactions between various cloud services (exposed over REST APIs) and snippets of user-defined compute logic. Hence, we propose a programming model in which applications are expressed as DAGs of *pure compute functions* and *communication functions*. Pure compute functions are snippets of untrusted user code that consume a declared set of inputs and produce a declared set of outputs. Communication functions enable interaction with external services. They are implemented by the Dandelion platform. Users invoke communication functions from user application DAGs, but for security, the implementation of communication functions cannot be modified by users. Currently, Dandelion supports an HTTP communication function that enables interaction with remote services like cloud storage buckets, AI inference services, and

data warehouses over REST APIs. Writing a Dandelion application involves: 1) expressing the DAG of compute and communication functions using a domain-specific language (DSL) and 2) providing the binaries for user-defined compute functions. §4 describes the DSL for expressing application DAGs (which also makes function inputs and outputs explicit), language support for developing user-defined compute functions, and the library of currently supported communication functions. This programming model is inspired by prior dataflow systems [47, 53, 79, 91, 118, 119]. Similar to systems like Spark [119], Dandelion's programming model makes data dependencies and thus data-parallelism and task-parallelism explicit to the platform. In contrast with traditional dataflow graphs, where edges encode internal communication and external communication is only performed at the dataflow sources and sinks, Dandelion function instantiations can either perform local compute or exchange data with other cloud services. Furthermore, we design Dandelion to execute dataflow DAGs with secure isolation in multi-tenant deployments without relying on execution in traditional VMs, which are slow to boot.

Decomposing a cloud application into pure compute and communication functions allows Dandelion to optimize the execution system for high elasticity while maintaining secure isolation guarantees (§5). It enables faster cold starts: pure compute functions can be securely executed in lightweight sandboxes that provide memory isolation (not necessarily VMs), since they do not rely on extra software environments like a guest OS. The separation of compute and communication also enables Dandelion to use cooperative network I/O and elastically adjust the CPU core allocation based on the number of compute vs. communication functions in the system. The platform can monitor each application's I/O intensity based on its number of compute vs. communication functions. Dandelion leverages this information for CPU core allocation, over-committing CPU cores for communication functions and dedicating CPU cores to compute functions.

Target applications. We design Dandelion for workloads with spiky load patterns (scaling from 0/few to 1/many with long periods of no/low load) or data-dependent parallelism. Many applications can benefit from Dandelion's ability to quickly scale-out to multiple CPU cores when an application computes on many independent inputs and quickly scale-in when the application needs to process data sequentially. Our goal is to enable fine-grain [66], elastic computing with secure isolation for a variety of cloud-native applications, such as data transformation pipelines (e.g., image compression, video transcoding [46, 103]), distributed log processing [62], CI/CD tasks, elastic query processing (which can include untrusted user-defined function (UDF) execution [89]), and emerging agentic AI workflows [39, 120].

Non-goals. However, Dandelion is not designed for applications that require large, frequent state synchronization, like online transaction processing, online gaming services,

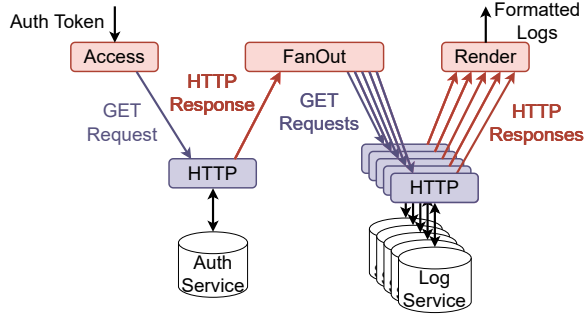


Figure 3. Dataflow for log processing example application.

AI training, or data processing algorithms that rely on fine-grain multithreading and shared memory for performance.

4 Dandelion Programs

4.1 User Interface

A complete Dandelion program (which we refer to as a “composition”) is a graph $G = (V, E)$, where: vertices V are one of (i) user-provided compute-only code, i.e., “compute functions”, (ii) Dandelion-provided facilities for I/O, i.e., “communication functions”, or (iii) other Dandelion programs, i.e., “compositions”. Directed Edges $E = (V_1, V_2, M)$ connecting the vertices show that one output set of V_1 is an input set of V_2 . M is a metadata descriptor, specifying which output and input set this edge describes as well as a keyword indicating how data is distributed to instances of V_2 . The keyword is one of ‘all’, ‘each’, or ‘key’, indicating if all input items in the set should be given to a single instance of V_2 , if each item should be given to a separate instance, or if items should be given to separate instances grouped by their keys. Keys are set by the user when formatting output data and are only used for grouping.

Figure 3 depicts an example Dandelion application that performs distributed log processing. The composition consists of three user-defined **compute functions** and multiple calls to the Dandelion **HTTP communication function**. The first compute function, **Access**, takes an input (e.g., an access token) and produces an output (e.g., an http request) that is then passed to the **HTTP** communication function provided by Dandelion. This function carries out the request and forwards the response (e.g., a list of authorized log service HTTP endpoints) to the **FanOut** function. **FanOut** function then takes this list and formats requests to retrieve the log files from each HTTP endpoint, dispatching them to **HTTP** functions in parallel. The last compute function, **Render**, aggregates the responses and templates them into a single output returned to the user. For more complex applications, compositions can include nested compositions, or spawn new compositions dynamically through Dandelion’s HTTP interface, e.g., to support dynamic control flow.

Dandelion provides a composition language to help users express DAGs of compute functions and communication functions in a more developer-friendly syntax. Listing 2 shows an example of the log processing example application expressed as a composition in Dandelion’s DSL. Our DSL is inspired by the domain-specific languages that other dataflow systems like Spark [119] and Timely [79] provide for users to specify DAGs. Our language could be replaced with other DSLs. Currently, users need to provide the composition and all compute function binaries directly, but ideally applications could be automatically decomposed into pure compute and communication functions, as we discuss in §4.3.

Communication Functions. Platform-provided communication functions enable user applications to interact with other services. They are designed to support high-level protocols with established user interfaces. A well-defined user interface enables the platform to safely validate requests to protect against malicious requests. It also frees the user from worrying about the lower-level implementation. Since communication functions are trusted, they are allowed to call system calls to establish and manage new connections. While Dandelion could support protocols such as gRPC or SCP, our current implementation focuses on HTTP, since cloud services generally expose REST APIs [86, 93]. Dandelion’s HTTP function first sanitizes inputs and then makes the specified GET/PUT/POST/DELETE requests, handing the response back to the composition for further processing.

Compute Functions. Compute functions can use familiar interfaces for memory allocation, local filesystem operations, and basic utilities like math functions, formatting, etc by linking to Dandelion’s custom libc (dlibc) or libc++ (dlibc++). These libraries provide a high-level interface with a userspace in-memory virtual filesystem. The in-memory filesystem represents function input sets and output sets as folders, with items as files within these folders. This enables a compute function to read inputs and write outputs as standard file operations without invoking system calls. Functions requiring system calls (e.g., mmap, mprotect, socket or threading) have stub implementations, returning appropriate error codes. Our current prototype provides SDKs for developing compute functions in C and C++, as well as a Python interpreter (see §4.2).

dlibc and dlibc++ run on top of a lower-level system interface. This interface uses a special data structure that Dandelion sets up before the function starts to point to descriptors for the input sets and items in the function’s memory. Before the function exits, it sets up the structure with the output sets and item descriptors. When a compute function uses the virtual file system to create outputs, dlibc automatically adds all files inside folders that represent output sets as output items.

```

1 auth_token = format_request(args[1])
2 auth_response = http_get(auth_url, auth_token)
3 authorized_servers = auth_processing(auth_response)
4 responses = []
5 for sever in authorized_servers:
6     responses.push(http_get(server))
7 rendered_html = <html template start>
8 for response in responses:
9     if response.successful():
10         rendered_html.append(reponse.render())
11     else:
12         rendered_html.append(response.error())
13 rendered_html.append(<html template end>)

```

Listing 1. Pseudocode for log processing application.

4.2 Language and Compilation Support

Dandelion provides software development kits (SDK) for functions written in C and C++. These SDKs help users compile their functions against `dlibc/dlibc++` or the low-level interface. To support Python functions, we have compiled the CPython interpreter using our C SDK. The same approach can be taken to support other interpreted languages with interpreters written in C or C++. To support more compiled languages, we aim to extend our SDK to be compatible with LLVM, such that any language with an LLVM frontend can be easily compiled for Dandelion. Additional languages can also be supported by Dandelion’s `rWasm` backend (§6.2), as it accepts Wasm code as input, which is a compilation target for many languages [34].

4.3 Porting Applications to Dandelion

Dandelion users currently need to express their applications as DAGs of pure compute and communication functions. Some applications like data analytics naturally fit this model, as query engines represent SQL queries as DAGs of operators. We also find that cloud-native applications like image or log processing pipelines that ingest data from cloud storage fit this programming model well. Based on our initial exploration with a Text2SQL agentic application in §7.7, Dandelion can also be suitable for some agentic AI workflows, which are an emerging class of applications that involve interactions between AI inference services, vector databases for retrieval, and custom user logic for pre- and post-processing data between these services.

However, users may also wish to port existing applications to Dandelion. To illustrate this process, consider the pseudocode in Listing 1 for the log processing example application from Figure 8. To port this application, the developer needs to be split the compute logic into separate functions between each of the two HTTP calls that the application issues. This results in three separate compute functions. The first compute function, called `Access`, contains the parsing logic for the input token and forms the request (line 1 from Listing 1). The second compute function, called `FanOut`, contains the logic from lines 3-5 in Listing 1 while the third

```

1 composition RenderLogs(AccessToken) => HTMLOutput {
2     Access(AccessToken = all AccessToken)
3     => (AuthRequest = HTTPRequest);
4     HTTP(Request = each AuthRequest)
5     => (AuthResponse = Response);
6     FanOut(HTTPResponse = all AuthResponse)
7     => (LogRequests = HTTPRequests);
8     HTTP(Request = each LogRequests)
9     => (LogResponses = Response);
10    Render(HTTPResponses = all LogResponses)
11    => (HTMLOutput = HTMLOutput);
12 }

```

Listing 2. Log processing application from Listing 1 expressed as a Dandelion composition DAG using our DSL.

compute function, called `Render`, contains the logic from lines 7-13. The HTTP requests issued on lines 2 and 6 in Listing 1 are done by communication functions in Dandelion. Listing 2 shows the composition that the user would write to express how Dandelion should pass data between compute and communication functions. The `each` keyword on both of the `HTTP` functions tells Dandelion it can parallelize the requests, while the `all` keywords on the compute functions indicate that all results of the preceding functions are collected and fed to a single instance of the function.

For applications that use asynchronous programming [88] (supported in popular languages like Python, Rust, and C++), we expect that Dandelion could use a compiler to automatically decompose an application into its pure compute and communication components by dividing the application into a graph of continuations [45]. Continuations are language-level primitives that can save the program state at arbitrary locations (e.g., at communication I/O boundaries) and resume execution at a later time. We plan to explore this approach as future work, drawing inspiration from `VectorVisor` [50], which uses continuations to port code containing system calls to run on GPUs.

4.4 Fault handling

When a communication function fails, it forwards the appropriate error code to the next downstream function(s). For example, an HTTP function may return 404 Not Found as a response. Dandelion compositions support conditional control flow, so failures can be handled gracefully. By default, functions are only executed if each of the input sets contains at least one item. This allows certain functions to only run if a failure message is present and to skip execution of further downstream functions if the failure cannot be recovered from. Sets that do not need to contain an item for the function to run can be marked as optional.

Dandelion’s failure handling could benefit from additional techniques, such as the log-based approaches in Boki [56], Beldi [84] or Halfmoon [84]. These systems can be used to find reentry points in compositions and skip functions that have successfully committed their progress.

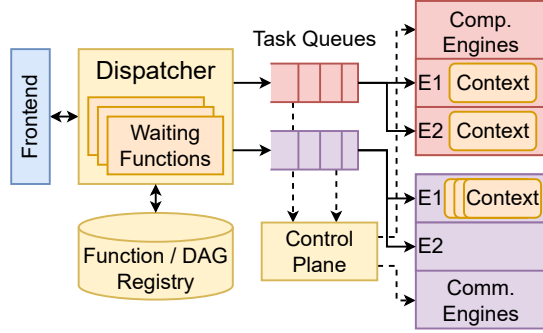


Figure 4. Dandelion worker node system architecture.

5 Dandelion Execution System

We now describe how Dandelion runs a composition and how we optimize the execution system for high elasticity. Figure 4 shows the Dandelion worker node system architecture. A client sends an HTTP request to the frontend.

HTTP frontend. The frontend manages client communication, handling requests for composition/function registration and invocation. It forwards these requests to the dispatcher and serializes and returns the final result to the client.

Dispatcher. The dispatcher orchestrates function execution within a Dandelion worker node. It keeps track of pending invocations, available compute and memory resources, and maintains a registry of all registered composition DAGs, function binaries, and associated metadata. The dispatcher schedules functions by tracking input/output dependencies and determines when a function is ready to run (i.e., when all its inputs are available).

The dispatcher manages memory and compute resources using two abstractions: *memory contexts* and *engines*, respectively. The dispatcher prepares an isolated memory context for each ready function and enqueues the function in a compute or communication engine queue for execution. When an engine finishes executing a function, the dispatcher associates its output data to the input of waiting functions. When a waiting function becomes ready, the dispatcher ensures that the outputs from prior functions are copied as inputs into the new function’s context. The dispatcher deallocates a completed function’s memory context when all data-dependent functions have consumed its output.

Memory contexts. A memory context is an abstraction for the dispatcher to manage the memory used by a function during execution. Each context is a bounded, contiguous memory region with methods to read or write at particular offsets and methods to transfer data to other contexts. Maintaining a simple context abstraction simplifies implementing different isolation mechanisms for compute functions, such as processes or lightweight VMs (§6). It also enables specializing methods for transferring data between different types of contexts. The maximum size of a context is set by the memory requirement specified by the user when registering

a function (like in AWS Lambda). Dandelion reserves this amount of virtual memory for the context and uses demand paging to allocate zeroed pages as needed.

Engines. Engines abstract compute resources that execute functions. Dandelion implements two types of engines: compute and communication engines. Each engine type polls a single, type-specific queue to ensure late binding of tasks. The dispatcher enqueues tasks (which consist of a prepared memory contexts and metadata) to the appropriate queue type and receives contexts containing the results.

Compute engines are responsible for securely executing untrusted user code. When a compute engine dequeues a task, it sets up the necessary isolation for the memory context and executes the function on the assigned compute resource. After execution completes, the engine inspects the left over context, updates metadata to indicate the locations of output data, and returns it to the dispatcher. §6 describes the implementation details for different isolation mechanisms. Compute functions do not block, so each compute engine only runs a single task at a time to completion² to minimize interference and context switching.

Communication engines, on the other hand, are part of the trusted execution system and therefore do not run in isolated sandboxes. These functions enable interactions (e.g., with HTTP requests) with external services by receiving a context with inputs and carefully validating these inputs to prevent corruption from potentially malicious users (see §6.3). They return a context with outputs from the function execution if input validation succeeds or an error if input validation fails. Unlike compute engines, communication engines use cooperative multi-threading with green threads to maximize efficiency.

Control plane. The worker control plane dynamically balances CPU resources between compute and communication engines to maximize application goodput. It periodically (every 30ms) measures the growth rates of the communication and compute engines’ queues. It uses the difference between their growth rates as an error signal for a Proportional-Integral controller [22]. If the control signal is positive, the control plane re-assigns a CPU core from the communication engine type to the compute engine type. If it is negative, it re-assigns a core from the compute engine type to the communication engine type.

Cluster manager. The cluster manager (not shown in Figure 4) orchestrates multiple worker nodes and load balances composition invocations across nodes. We extended Dirigent [38] to support Dandelion worker nodes, but other cluster managers could also be used.

²Tasks that run for longer than a user-specified timeout (e.g., long or infinite loops) will be preempted to prevent resource hogging.

6 Implementation

We implement Dandelion in ~12K lines of Rust. We describe how the dispatcher orchestrates execution (§6.1), how compute engines provide isolation with support for four different memory isolation mechanisms (§6.2), and how we implement communication engines (§6.3).

6.1 Dispatcher and Engine Coordination

The dispatcher orchestrates composition invocations using separate green threads [8]. It queues functions as their inputs become available and coordinates data movement. To avoid bottlenecks, the dispatcher offloads tasks that are not part of the control flow, such as loading code into contexts, transferring data between contexts, and executing functions.

Data passing. To move data between contexts, Dandelion currently copies data. However, this is not fundamental to the programming model. Different backends could avoid the copy by remapping memory or using copy-on-write when code or data is needed by multiple functions, if the security model allows for it. This is future work.

6.2 Compute Engines

Compute engines execute compute functions with one of four memory isolation options. We implement different isolation backends to demonstrate that Dandelion’s design is not tied to a particular mechanism. Cloud providers can select a mechanism based on its performance and security tradeoffs, which we evaluate in §7 and §8.

KVM-based isolation. The KVM backend leverages hardware support for virtualization (e.g. Intel VT-x) to isolate compute functions. Each compute function runs as a lightweight VM without a guest kernel, as we don’t need to support syscalls. To minimize overheads we use an identity mapping between guest virtual and guest physical addresses. The engine is a minimal hypervisor, which is only responsible for VM launch and exit using Linux’s Kernel-based Virtual Machine (KVM) module [85]. KVM relies on hardware nested paging (e.g. EPT in VT-x) for memory safety. The memory context is an anonymous memory mapping in the Dandelion process, that is used to back the guest physical address space. Inspired by Virtines [109], we circumvent VM setup cost by reusing KVM structures. For this, we reset virtual CPU state and replace the memory backing the guest physical address space, effectively switching the context.

Process-based isolation. The process-based isolation backend executes each function as a separate process. Dandelion uses a shared memory region as context. When a function is ready to execute, the engine that picks up the task spawns a new process. The process first runs a small amount of trusted code to map the shared memory into the process. The new process sends a ready signal to the engine, which uses ptrace to catch and prevent any system calls the compute function may attempt to make. If a syscall is caught,

Dandelion will terminate the function and notify the user. If the function process exits regularly, the engine parses the data left behind in the shared memory to extract the output of the function.

CHERI-based isolation. Dandelion can also leverage new CPU hardware security extensions, like CHERI [110]. With CHERI, load/store instructions take capabilities as inputs rather than integer addresses. Instructions succeed if the capability passes permission and bounds checks performed by the CPU hardware, otherwise it faults.

Dandelion’s CHERI backend can run compute functions as threads within the Dandelion process, since CHERI capabilities enforce isolation within a virtual address space. We use CHERI’s hybrid capability mode [110] to avoid requiring capability compatibility from all user code. CHERI contexts are contiguous memory regions, pointed to by a code and a data capabilities. An engine sets these capabilities as default capabilities and ensures it no longer has any other capabilities before jumping into user code. Special sealed capabilities are used to return from user code, which are guaranteed by hardware to not have been manipulated.

rWasm-based isolation. Dandelion can also use isolation mechanisms that do not require hardware support. We show this by implementing a backend using rWasm [27], a Wasm-to-safe-Rust compiler. Developers register compute functions in Wasm code. Dandelion transpiles the user-defined Wasm code to safe Rust code, then wraps and compiles it to a native shared library. To execute functions, Dandelion only needs to load the library and call into it using the symbols defined in the wrapper. Memory isolation is provided by the memory safety guarantees for safe code by the Rust compiler, which is already part of the trusted computing base as Dandelion is implemented in Rust. Since memory isolation is enforced at compile time, this approach does *not* rely on a runtime like Wasmtime for isolation.

6.3 Communication Engines

Each communication engine runs a separate kernel thread pinned on a dedicated core, which executes its own asynchronous runtime, using green threads to run multiple requests in parallel. Communication engines have the same interface to the dispatcher as compute engines and similarly poll task queues. Since the communication engines are trusted, they do not need to execute functions in sandboxes, but they do need to sanitize input data, as this is untrusted. For our HTTP function, we only rely on the first line defined by the protocol to contain the HTTP method and protocol version. Dandelion can check these fields against a fixed set of options and the first part of the URI, which identifies the host to connect to with either a valid IP or a domain name.

7 Performance Evaluation

We evaluate Dandelion’s sandbox creation (§7.2) and function execution latency and throughput (§7.3) with different isolation backends. We investigate the impact of executing an application as a composition of compute and communication functions in Dandelion versus as single function with Firecracker or Wasmtime (§7.4). We also show that Dandelion’s separation of compute and communication functions enables more efficient CPU scheduling (§7.5). Finally, we show Dandelion’s end-to-end performance benefits when multiplexing real applications with different compute and I/O intensities (§7.6), more complex workflows for elastic query processing (§7.7), and its memory savings when running the Azure Functions production trace (§7.8).

7.1 Methodology

Hardware configurations. We run experiments on several CPU platforms. Our default machine is a dual-socket Intel Xeon E5-2630v3 with 16 physical cores, 256GiB of DRAM, and a Mellanox ConnectX-3 NIC. We also use this type of machine to run the loader. For experiments with the CHERI backend, we run Dandelion on an Arm Morello board [2] that implements CHERI capabilities on top of the aarch64 instruction set with a 4-core processor, 16GiB of RAM, and a Mellanox ConnectX-3. We disable frequency scaling for all experiments. For Azure trace experiments, we use Cloudlab [3] d430 nodes, which have dual-socket Intel E5-2630v3 CPUs with a total of 16 physical cores and 64GB of DRAM.

Baselines. We compare Dandelion to Firecracker [16], the open-source MicroVM hypervisor used by AWS Lambda. We use a simple HTTP relay to route requests to MicroVMs and spawn new MicroVMs. When the relay receives a request for a hot function, it passes it to the first available MicroVM. When it gets a cold request, it boots a new MicroVM and forwards the request on as soon as that VM is ready. We run Firecracker (which we abbreviate as FC) with and without snapshots [5]. The Firecracker baseline represents VM-based isolation approaches. Dandelion’s most similar backend for comparison is the KVM backend, as it relies on the same hardware virtualization.

We use the same HTTP relay setup for gVisor [6], which uses containers instead of MicroVMs. The most comparable isolation backend in Dandelion is the process backend, as it also uses processes with restricted capabilities.

We also run Spin [4], an open-source framework for building and running event-driven applications with Wasmtime [17]. We use Spin’s recommended default settings, which include enabling pooled allocation with 1000 Wasmtime instances. For simplicity, we refer to the Spin/Wasmtime setup as Wasmtime or WT. This baseline represents software fault isolation and is most suitable to compare to Dandelion’s rWasm isolation backend.

	CHERI	rWasm	process	KVM
Marshal requests	12	15	12	30
Load from disk	29	147	54	194
Transfer input	2	2	6	2
Execute function	5	20	371	536
Get/send output	9	12	9	25
Other	32	45	34	102
Total	89	241	486	889

Table 1. Dandelion avg. latency breakdown in μ s for each isolation backend running 1x1 matmul on Morello.

Software setup. We run Ubuntu 22.04. On the Morello board, we use a modified kernel with Morello support based on version 6.7.0, with programs running in CHERI hybrid mode. We use Firecracker 1.5.0, a Firecracker quickstart guest kernel, and a rootfs built on top of Alpine [1] with only necessary packages added. For gVisor, we use the 20240715 release with an Alpine Linux base image. For best performance, we choose the KVM platform for gVisor’s syscall interception. For Spin/Wasmtime, we use Spin version 3.0 configured with its recommended defaults and Wasmtime 25.0.3. We use Dirigent [38] as the cluster manager for Azure trace experiments.

7.2 Sandbox Creation

Table 1 shows the unloaded latency breakdown for Dandelion sandbox creation with each isolation backend. The CHERI backend performs best as it executes within a single address space without spawn new threads on the critical path. The main work is loading the executable, parsing it, and moving it to the correct place in memory (*load from disk* in Table 1). The rWasm backend, using software isolation, achieves fast function execution but is mainly limited by slow dynamic loading. The process backend spends most of the time creating and preparing new processes (*execute function* in Table 1), while KVM backend is slowed by lightweight VM setup, worse on Morello with a CHERI-compatible kernel. When using the default Linux 5.15 kernel, the total latencies of the rWasm, process, and KVM backends are 109, 539, and 218 microseconds, respectively.

We compare Dandelion’s sandbox creation tail latency to other systems as we sweep throughput in Figure 5, using 1x1 64-bit integer matrix multiplication with 0% hot invocations. Creating a sandbox in Firecracker (FC) involves booting a fresh MicroVM, which takes over 150ms. FC can reduce startup latency by loading pre-initialized MicroVM state from a snapshot on disk instead of initializing the guest OS and application on the critical path. However, snapshot restoration still involves creating a MicroVM and restoring its state, limiting it to 120 RPS. gVisor [6] uses a hardened container instead of a MicroVM. As gVisor performed worse than FC with snapshots, we omit it in remaining plots. Initializing a Spin/Wasmtime (WT) instance is lightweight due to

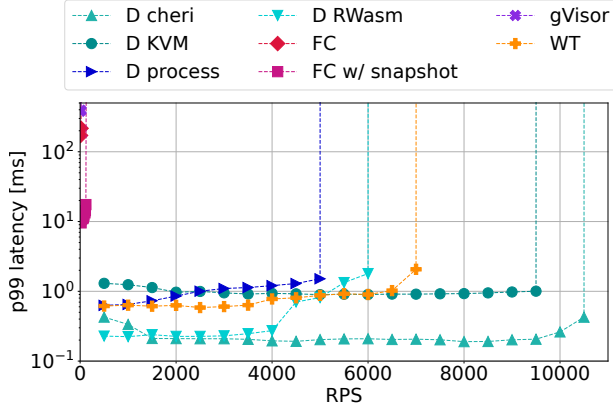


Figure 5. Tail latency vs. throughput for 1x1 matmul on the Morello server, with 0% hot requests.

multiple optimizations, including pooling memory allocation and pre-instantiating Wasm components, which allows WT to achieve 7000 RPS peak throughput. Although not shown in the figure, we also experimented with Microsoft’s recent Hyperlight Wasm [77] system, using their default sandbox configurations. We measure 9.1 ms average unloaded cold start latency with Hyperlight Wasm, which includes launching a ProtoWasmSandbox (2.8ms), loading the Wasmtime runtime (4.2ms) and loading the Wasm module (2.1ms). We also considered Unikraft [65], but did not run separate experiments. Their paper reports a 3.1ms VM boot time to user space *main* function using Firecracker as VMM. This excludes the time to receive a request and send a response, which would be included in our experiment, making its performance very similar to FC with snapshots.

7.3 Compute Function Performance

Figure 6 shows a compute microbenchmark (128x128 64-bit integer matrix multiplication) on the default server setup. We conservatively use a 97% hot request ratio for Firecracker experiments, based on Shahrade et al. [95] observing that 3.5% of applications in the Azure Functions trace experience *only* cold starts, even if sandboxes are never torn down. We plot median latency with 5th and 95th percentile error bars to show the impact of cold starts on overall performance. Dandelion and Wasmtime create a new sandbox per request. For Dandelion, we load the function binary from disk instead of in-memory cache for 3% of requests. Dandelion shows low, stable latency, peaking at 4800 RPS with the KVM backend, but its rWasm backend suffers from slower matrix multiplication code due to transpilation.

Firecracker shows similarly low unloaded latency by committing extra memory for hot MicroVMs, ensuring that most requests do not require booting a new sandbox. However, its latency gradually increases and saturates at 3000 RPS (with snapshots), becoming increasingly unstable after 2800 RPS due to CPU contention between active MicroVMs and

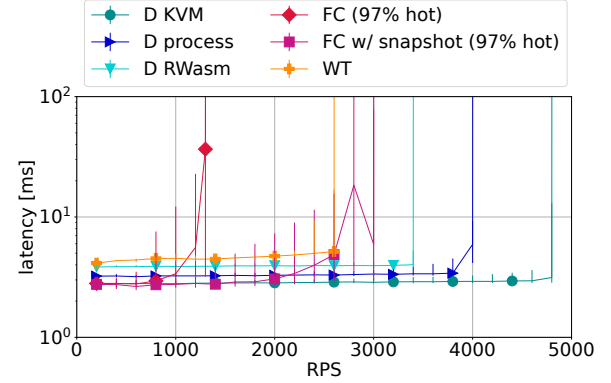


Figure 6. Compute function (128x128 matmul) on 16-core server. Latency is median, with 5th / 95th percentile error bars.

MicroVM creations. Wasmtime, despite its fast sandbox creation, has higher unloaded latency and saturates at only 2600 RPS due to the compiler toolchain’s less efficient code generation and optimization. To support 128x128 matrix multiplication in Hyperlight Wasm, we increase default configuration `guest_input_buffer`, `guest_output_buffer`, and `guest_stack_size` to 1 MB each, and `guest_heap_size` to 2 MB, which is the minimum required to avoid out of memory errors. Hyperlight Wasm has a high unloaded average latency of 27.5ms, including sandbox creation (2.6ms), runtime loading (12.1ms), module loading (4.7ms), and function execution (8.1ms).

7.4 Composition Performance Overhead

While §7.2 showed performance of individual sandbox creation, decomposing applications into compute and communication functions can increase the number of sandboxes that need to be created. To evaluate this tradeoff, we design a microbenchmark that fetches a 64KiB array and computes sum, min and max over a sample of the elements; we call this sequence a phase. We sweep the number of phases in the microbenchmark from 2 to 16.

We compare Dandelion KVM with Wasmtime, Firecracker hot (VM already running) and Firecracker with cold start from snapshots. For Dandelion, we consider loading the compute function code from disk (uncached) and copying from memory (cached) to see how keeping code ready in RAM impacts chains of functions. All systems show linear growth in latency as we increase the number of compute-communication phases. Firecracker hot scales best, but Dandelion KVM uncached is only 1.3ms (17%) slower than Firecracker hot at 8 phases and 4ms slower at 16 phases. Notably, Dandelion KVM cached vs. uncached differs by just 0.5ms at 16 phases, indicating disk loading is competitive even for long chains. Wasmtime has the same slope as Dandelion KVM, hinting at potentially worse application scaling on Wasmtime, which is balanced out by the repeated sandbox creations in Dandelion in this experiment. Firecracker cold

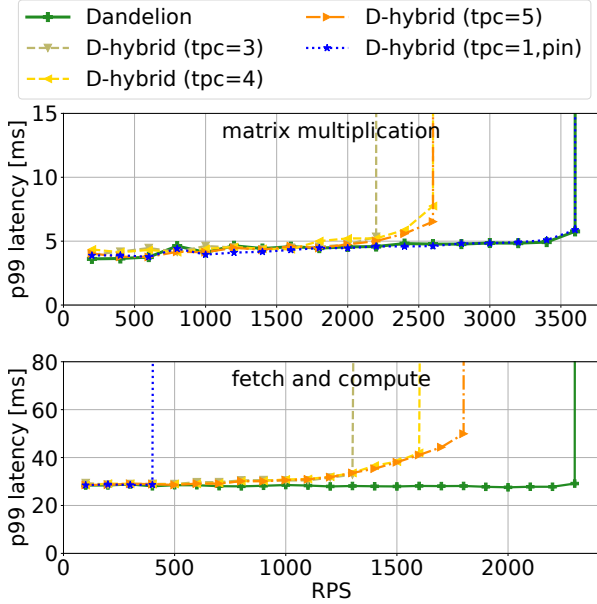


Figure 7. Separating compute and communication function execution in Dandelion vs. unifying in D-hybrid for two types of applications (compute-intensive and I/O intensive).

start from snapshots has higher base latency but same latency slope as Firecracker hot. Nonetheless, Dandelion KVM uncached is 4.6x faster at 16 phases. Overall, our results show that Dandelion still provides great performance even for applications requiring numerous sandbox creations.

7.5 Throughput Benefits of Compute-Comm Split

In this section, we highlight an additional benefit Dandelion gains from splitting compute and communication: more efficient core multiplexing. In traditional FaaS, each sandbox runs its own networking, and the OS scheduler multiplexes sandboxes across CPU cores, incurring costly context switches. Determining the optimal number of sandboxes to execute concurrently per node to maximize application throughput is hard. Compute-intensive sandboxes benefit from low CPU oversubscription, while I/O intensive ones need high oversubscription to fully utilize the CPU resources. Moreover, sandboxes can switch between being I/O intensive and compute intensive at any time.

Dandelion’s compute-communication split enables cooperative networking, run-to-completion for compute functions, and efficient reallocation of CPU cores between compute and communication functions. To measure the impact of Dandelion’s programming model, while keeping the rest of the system the same, we implement Dandelion-hybrid (D-hybrid). It uses the same system architecture and isolation backends as Dandelion, but supports running a composition as a single “hybrid” function, allowing opening sockets for communication.

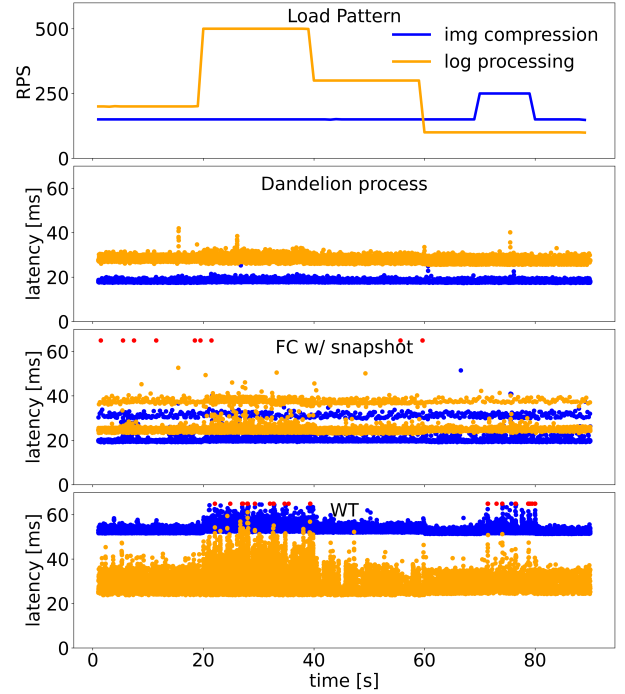


Figure 8. Multiplexing compute vs. I/O-intensive apps, with changing RPS over time. Dandelion dynamically adjusts compute/comm engines to maintain low latency. Red dots show requests where latency exceeds 65ms.

In Figure 7, we run the compute-intensive (matrix multiplication) and I/O-intensive (fetch and compute) microbenchmarks used in §7.4, varying 1 to 8 threads per CPU core (tpc), with and without core pinning for D-hybrid. We present the highest-performing configurations in either microbenchmark (3-5 tpc without pinning and 1 tpc with pinning). Depending on the compute and I/O intensity of the applications, D-hybrid requires fundamentally different levels of concurrency to maximize throughput (i.e. 1 tpc pinned for matrix multiplication and 5 tpc unpinned for fetch and compute). Designing a controller to adjust concurrency based on system load metrics like CPU utilization is non-trivial: matrix multiplication achieves peak throughput at 90% CPU utilization with tpc 1 with pinning, while fetch and compute achieves peak throughput at 70% CPU utilization with tpc 5 without pinning.

In contrast, Dandelion’s control plane achieves the highest throughput for both workloads. Dandelion delivers even higher throughput and consistently lower tail latency for the I/O-intensive application by running compute tasks to completion on dedicated cores and aggregating cooperating communication tasks on the remaining cores, minimizing context switches. In the next section, we further show how Dandelion multiplexes requests from multiple applications with different compute and I/O intensities.

7.6 Multiplexing Applications

While previous experiments showed that Dandelion outperforms Firecracker and Wasmtime with compute and composition microbenchmarks, we now evaluate its behavior in a more realistic setting where the system needs to manage mixed workloads with bursty traffic.

In Figure 8, we run the distributed log processing application from Figure 3 (I/O-intensive) and an image compression application that transforms an 18kB QOI [100] image to PNG (compute-intensive) on Dandelion, Firecracker (with snapshot), and Wasmtime under a bursty load pattern. Image compression is as an example of image processing and log processing is an example of HTML templating, which are both common task from FaaS benchmarks [35, 60, 108]. Consistent with previous experiments, we use a 97% hot request ratio for Firecracker.

Firecracker has a bi-modal latency distribution for both applications. This is because 97% of requests are served by hot VMs, which benefit from pre-warmed state, while the remaining 3% incur cold-start (snapshot restoration) overhead. By overprovisioning memory to maximize warm starts, Firecracker achieves a relatively low average request latency for both compression (20.4ms) and log processing (25.6ms), but has high variance at 389% and 1495% respectively due to the cold starts. Spin/Wasmtime, on the other hand, struggles to handle the mixed workload effectively, increasing the latency of both applications. As discussed in §7.3, Wasmtime suffers from less efficient code generation and optimization. Spin's reliance on the Rust Tokio async runtime also exacerbates interference between applications [14, 73]. Spin schedules functions cooperatively, meaning a thread can be occupied by a compute intensive function for an arbitrary amount of time. We observe that image compression tasks hog resources, causing log processing tasks to queue behind them, increasing tail latency. While this keeps the variance of the compression application relatively low at 6.1%, this is not a good tradeoff as the average latency is 53.3ms. The log processing application has a 79.2% variance with an average latency of 28.9ms.

Dandelion maintains stable latency for both applications, even during load bursts, due to fast cold starts and the controller's dynamic re-allocation of CPU cores, e.g., it scales from a single I/O core at the beginning of the experiment to 4 I/O cores to accommodate I/O-intensive log processing during peak load. For the image compression application, Dandelion achieves an average latency of 18.2ms and p99 latency of 19.6ms, which is 38.6% lower than Firecracker and 67.1% lower than Wasmtime. For the log processing application, Dandelion delivers an average latency of 27.9ms and p99 latency of 30.6ms, which is 19.5% lower than Firecracker and 33.5% lower than Wasmtime. Dandelion reduces performance variability, achieving the lowest relative variance of 1.3% for image compression and 2.9% for log processing.

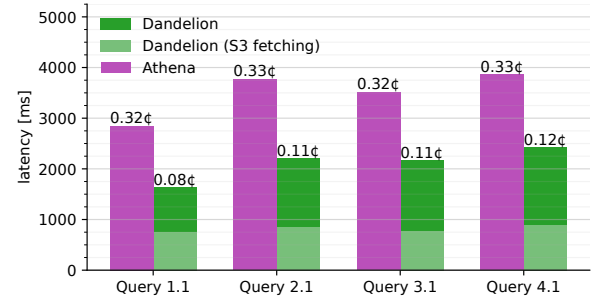


Figure 9. SSB query latency and cost in US cents compared to Athena. The queries ingest ~700MB input data from S3.

Overall, Dandelion efficiently multiplexes requests with different compute and I/O intensities and maintains low and stable latency despite creating a sandbox on the critical path for each request.

7.7 Elastic Data Processing with Dandelion

We now explore Dandelion's suitability for a broader range of target applications with more complex workflows.

Serverless query processing. Query-as-a-Service (QaaS) offerings like AWS Athena [13] enable SQL-based data analysis on cloud storage, with providers managing infrastructure and billing per byte read. We show that Dandelion improves QaaS elasticity. Figure 9 shows query execution latency and cost with Athena and Dandelion for Star Schema Benchmark [80] queries (which are based on the industry standard TPC-H [37] benchmark) using 700MB of input data. The queries include filter, projection, join, order by, and aggregation operators, which we implement in Dandelion by porting the Apache Arrow Acero [12] library operators and linking against `h11bc++`. We run Dandelion on an EC2 m7a.8xlarge instance in the same region as the S3 bucket and compute cost based on the execution time and EC2 VM price. Athena charges per bytes read and we exclude the queuing delay in the plot. Dandelion quickly boots sandboxes and spreads query execution across all 32 CPU cores, achieving 40% lower latency and 67% lower cost than Athena. We expect Dandelion's secure isolation of query operators to be particularly useful for queries that embed user-defined functions (UDFs), as these contain arbitrary and untrusted user code that needs to be sandboxed in multi-tenant environments [89]. For example, AWS Athena executes UDFs in AWS Lambda. While this experiment does not compare the two systems running on the same hardware or with the same operator implementations, we include it to compare the latency and cost of a commercially available service with what we can achieve using Dandelion as the underlying QaaS system on EC2 VMs. With larger input data sizes (we tested up to 7GB), matching Athena's latency requires scaling query execution across multiple Dandelion nodes, but we continue to see lower query execution cost compared to Athena.

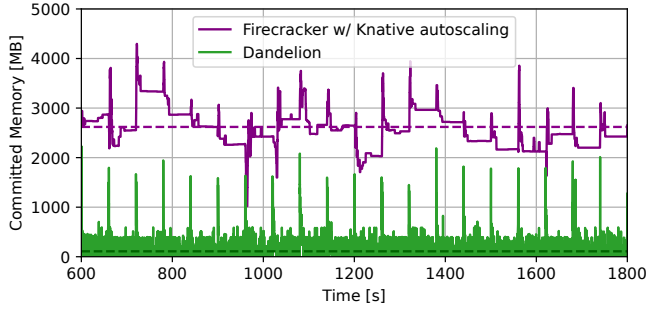


Figure 10. Memory committed over time for Azure trace experiment. Dotted lines show the average.

Agentic AI query processing workflow. We explore Dandelion’s suitability for emerging agentic AI applications, which consist of multiple interacting components, including model inference services, retrievers, and external tools and often require pre- and post-processing compute logic between these calls [30, 39]. We target Text2SQL workflows, a popular subset of agentic AI application, converting natural language queries to SQL. A Masters student with no prior experience with Dandelion ported a Text2SQL workflow from the TAG benchmark suite [25] in a few hours. The workflow processes a sample prompt in ~ 2 s with five steps: 1. parsing the input prompt (221 ms), 2. requesting an LLM with the prompt via HTTP (1238 ms), 3. extracting the SQL query from the LLM’s response (207 ms), 4. issuing the SQL query via HTTP to a SQLite database (136 ms), and 5. formatting the database response (213 ms). We implement the parsing, SQL extraction, and formatting as Dandelion Python compute functions, while the LLM and database queries are performed by Dandelion communication functions. We use Gemma-3-4b-it running on a single H100 NVL as our LLM inference service exposed over a REST API. The LLM inference step is the pipeline’s bottleneck, accounting for 61% of the total end-to-end latency.

7.8 Azure Functions Trace Evaluation

Finally, we compare the performance and memory efficiency of Dandelion (with process isolation backend) and Firecracker, while running the Azure Functions trace sample in the Cloud-Lab setup. We sample 100 functions from day 6, hour 8 of the trace using the InVitro [105] sampler. We use the autoscaling policy in Knative [7], a popular open-source FaaS orchestrator, to control the number of Firecracker MicroVMs over time based on application load. We find that Knative scales MicroVMs for this trace such that on average 3.3% of invocations are cold. This validates our 97% hot requests setting in prior experiments.

Figure 10 shows the memory committed by Firecracker with Knative autoscaling compared to Dandelion over a 20-minute long trace. To minimize cold starts, Firecracker maintains MicroVMs long after a request has finished executing,

which requires committing memory for long periods. In contrast, Dandelion commits and consumes memory only while requests are actively running since a new context is created for each request. This enables Dandelion to commit only 4% of the memory Firecracker commits on average (109 MB vs. 2619 MB), achieving significant cost savings and comparable or even lower end-to-end response time for functions. Dandelion also reduces the p99 end-to-end latency by 46% compared to Firecracker in this experiment.

8 Security Analysis

To compare the security implications of Dandelion and systems like Firecracker, gVisor, and Wasmtime, we analyze their attack surface and trusted computing base (TCB).

Threat model. We assume users trust the cloud platform, but not other users. The cloud provider does not trust users. Users do not have physical access to machines, but can exploit vulnerabilities in cloud software [31, 68] to breach the confidentiality and integrity of other users’ applications. Denial of service and side-channel attacks are out of scope; we assume the provider deploys necessary defenses [99].

Attack surface. We analyze the vulnerability of the interface that untrusted functions can directly access. Most FaaS platforms provide user functions a POSIX-like environment with various system calls available. To mitigate this large attack surface [114], Firecracker and gVisor rely on defense-in-depth strategies, running system calls for different functions with separate OS kernels. Wasmtime offers a more controlled attack surface via WASI [10]. Dandelion goes further in this direction: users provide only pure compute functions and system calls are blocked during their execution (§4.1). Although a malicious compute function could produce output data that attempts to hijack control flow when parsed by the dispatcher, Dandelion’s function output parser is merely 100 lines of Rust, making it feasible to verify its memory safety and correctness. Users may attempt to exploit communication functions by passing malicious inputs. Dandelion’s communication engines guard against such attacks by checking that each input is a valid HTTP protocol request.

Trusted computing base. Dandelion, Firecracker and Spin have several TCB components in common: the Linux kernel, C standard library, and Rust compiler. While the Firecracker and Spin code base has expanded to ~ 68 k and ~ 65 k lines of Rust code respectively, Dandelion consists of ~ 12 k lines of Rust code including tests, with only 2k lines directly relevant to isolation and interaction with user code. gVisor has ~ 38 k lines of Go, excluding third-party packages.

9 Related Work

Serverless programming models. Dandelion’s design is closely related to other dataflow paradigms for serverless. Computation-centric networking [41] involves a separation

between I/O and compute, with delineated nondeterminism. Lambdata [102] and Dataflower [71] leverage application dataflow information to reduce function triggering latency and avoid round-trips to remote storage for data exchange. Pheromone [117] exposes a data bucket abstraction (for intermediate function outputs) and trigger primitives, which allows developers to specify when and how to pass intermediate data to other functions and invoke them. Other FaaS systems introduce data dependency hints to optimize function placement across nodes [15, 70, 75]. Nu [87] provides a new process abstraction that decomposes an application into fine-grained units of state and compute, which can be quickly migrated to improve resource utilization. However, Nu only provides process-level isolation, which is not sufficient for multi-tenant cloud environment.

SigmaOS [101] forgoes POSIX compatibility and designs a cloud-centric communication API to improve container performance for serverless and microservice tasks. However, SigmaOS still allows user code to invoke 67 host system calls, which has different security implications than Firecracker, gVisor, and Dandelion, which restrict user code from making direct system calls to the host kernel.

Lightweight isolation mechanisms. Complementary to Dandelion's four isolation backends, Wedge [26] introduces sthreads for intra-process privilege separation and memory isolation, Shreds [32] isolates code and data compartments using Arm memory domains, and lwCs [72] are independent units of protection, privilege, and execution state within a process. Virtines [109] is a programmer-guided abstraction allowing individual functions within an application to run in lightweight, virtualized execution environments. X-containers [96] is an exokernel-inspired container architecture designed to isolate single-concerned cloud-native applications without nested hardware virtualization. GraalOS [81] and V8 [106] isolate code with language runtimes.

Data processing systems. DAG-based systems for data processing such as Spark [119] are already popular. These systems usually focus on single users or groups of mutually trusting users. Dandelion innovates on the DAG model for processing, as it leverages it to achieve secure isolation and high elasticity in multi-tenant environments.

10 Conclusion

The cloud has evolved in the past decade to offer a variety of serverless compute, storage, and AI/analytics services, but the performance and efficiency of these services is limited by the legacy software that they retrofit. Today's infrastructure is still based on the older, more traditional model of cloud computing, in which users rent long-running VMs, each with a guest OS exposing a POSIX-like interface. Initializing this system stack is slow. Furthermore, having applications interact with the platform on the level of files and sockets does not provide the platform with useful information about

application characteristics, like compute vs. I/O intensity, to optimize resource allocation and scheduling.

Dandelion co-designs a declarative cloud-native programming model and execution system for truly elastic computing while maintaining secure isolation. Developers specify their applications as DAGs of pure compute functions and communication functions. This enables Dandelion to isolate pure compute functions with lightweight sandboxes that boot in 100s of microseconds, enabling true elasticity (i.e., fast cold start for every request). Dandelion improves elastic query processing latency by 40% compared to AWS Athena and reduces committed memory by 96% on average compared to Knative autoscaling for the Azure Functions trace.

11 Acknowledgements

We would like to thank Gustavo Alonso, Timothy Roscoe, Xiaozhe Yao, and Maximilian Böther for their helpful insights during discussions. We would also like to acknowledge Roberto Starc, Lorenzo Rosa, Georgijs Vilums, and Josef Schönberger, and Alessio Russo for their contributions to the project. Thank you to Eric Van Hensbergen for his inputs in the early stages of the project and his support with the Arm Morello setup. We also thank our anonymous reviewers from ASPLOS'25 and OSDI'25 for their feedback and suggestions. Tom Kuchler, Pinghe Li, and Yazhuo Zhang are supported by the Swiss National Science Foundation (project TMSGI2_218019).

References

- [1] Alpine Linux. Available at <https://alpinelinux.org/>.
- [2] Arm Morello Program. <https://www.arm.com/architecture/cpu/morello>.
- [3] Cloudlab. <https://www.cloudlab.us/>.
- [4] Fermion spin. <https://github.com/fermyon/spin>, 2024.
- [5] Firecracker snapshotting. Available at <https://github.com/firecracker-microvm/firecracker/blob/main/docs/snapshotting/snapshot-support.md>, 2024.
- [6] gVisor. Available at <https://gvisor.dev/docs>, 2024.
- [7] Knative. Available at <https://knative.dev>, 2024.
- [8] Rust Module tokio::task. Available at <https://docs.rs/tokio/latest/tokio/task/index.html>, 2024.
- [9] WebAssembly. <https://webassembly.org>, 2024.
- [10] WebAssembly System Interface. <https://wasi.dev/>, 2024.
- [11] Amazon managed workflows for apache airflow (mwa). Available at <https://aws.amazon.com/managed-workflows-for-apache-airflow/>, 2025.
- [12] Apache Arrow Acero Overview. Available at <https://arrow.apache.org/docs/cpp/acero/overview.html>, 2025.
- [13] AWS Athena. <https://aws.amazon.com/athena/>, 2025.
- [14] Rust tokio runtime. <https://docs.rs/tokio/latest/tokio/runtime/index.html>, 2025.
- [15] Mania Abdi, Samuel Ginzburg, Xiayue Charles Lin, Jose Faleiro, Gohar Irfan Chaudhry, Inigo Goiri, Ricardo Bianchini, Daniel S Berger, and Rodrigo Fonseca. Palette Load Balancing: Locality Hints for Serverless Functions. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 365–380, 2023.

- [16] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020.
- [17] Bytecode Alliance. Wasmtime. <https://wasmtime.dev/>, 2024.
- [18] Gustavo Alonso, Ana Klimovic, Tom Kuchler, and Michal Wawrzoniak. Rethinking Serverless Computing: from the Programming Model to the Platform Design. In *VLDB Workshop on Serverless Data Analytics*, 2023.
- [19] Mohamed Alzayat, Jonathan Mace, Peter Druschel, and Deepak Garg. Groundhog: Efficient Request Isolation in FaaS. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 398–415, 2023.
- [20] Amazon Web Services. The Security Design of the AWS Nitro System. <https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/security-design-of-aws-nitro-system.html>, 2022.
- [21] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and Matei Zaharia. A view of cloud computing. *Commun. ACM*, 53(4):50–58, April 2010.
- [22] Karl Johan Åström and Tore Hågglund. Controller design. In *PID Controllers: Theory, Design, and Tuning*, chapter 4, pages 120–156. Instrument Society of America, 2 edition, 1995.
- [23] AWS. What is RESTful API? <https://aws.amazon.com/what-is/restful-api/>, 2025.
- [24] Microsoft Azure. What are Durable Functions? <https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview?tabs=csharp-inproc>.
- [25] Asim Biswal, Liana Patel, Siddharth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. Text2SQL is not enough: Unifying AI and databases with TAG. <https://arxiv.org/abs/2408.14717>, 2024.
- [26] Andrea Bittau, Petr Marchenko, Mark Handley, and Brad Karp. Wedge: Splitting Applications into Reduced-Privilege Compartments. In *5th USENIX Symposium on Networked Systems Design and Implementation (NSDI 08)*, 2008.
- [27] Jay Bosamiya, Wen Shih Lim, and Bryan Parno. Provably-Safe Multilingual Software Sandboxing using WebAssembly. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022.
- [28] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems, EuroSys '20*, New York, NY, USA, 2020. Association for Computing Machinery.
- [29] Paul Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. The Rise of Serverless Computing. *Commun. ACM*, 62(12):44–54, 2019.
- [30] Gohar Irfan Chaudhry, Esha Choukse, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. Towards resource-efficient compound ai systems, 2025.
- [31] Haogang Chen, Yandong Mao, Xi Wang, Dong Zhou, Nickolai Zeldovich, and M. Frans Kaashoek. Linux Kernel Vulnerabilities: State-of-the-Art Defenses and Open Problems. In *Proceedings of the Second Asia-Pacific Workshop on Systems, APSys '11*, New York, NY, USA, 2011. Association for Computing Machinery.
- [32] Yaohui Chen, Sebasujeen Reymondjohnson, Zhichuang Sun, and Long Lu. Shreds: Fine-Grained Execution Units with Private Memory. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 56–71, 2016.
- [33] Alvin Cheung, Natacha Crooks, Joseph M. Hellerstein, and Mae Milano. New Directions in Cloud Programming. In *11th Conference on Innovative Data Systems Research, CIDR 2021, Virtual Event, January 11–15, 2021, Online Proceedings*. www.cidrdb.org, 2021.
- [34] Emscripten Contributors. Emscripten: An LLVM-to-WebAssembly Compiler. <https://emscripten.org/index.html>, 2015.
- [35] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefer. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. *arXiv:2012.14132 [cs]*, July 2021.
- [36] Cloudflare. Cloudflare workers security model. <https://developers.cloudflare.com/workers/reference/security-model/>, 2024.
- [37] Transaction Processing Performance Council. TPC-H. <https://www.tpc.org/tpch/>, 2025.
- [38] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. Dirigent: Lightweight Serverless Orchestration. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP '24*, page 369–384, New York, NY, USA, 2024. Association for Computing Machinery.
- [39] Databricks. What are compound AI systems and AI agents? Available at <https://docs.databricks.com/en/generative-ai/agent-framework/ai-agents.html>, 2024.
- [40] Datadog. The State of Serverless 2021. <https://www.datadoghq.com/state-of-serverless-2021/>.
- [41] Yuhang Deng, Angela Montemayor, Amit Levy, and Keith Winstein. Computation-Centric Networking. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks, HotNets '22*, 2022.
- [42] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '20*, page 467–481, New York, NY, USA, 2020. Association for Computing Machinery.
- [43] Nafise Eskandani and Guido Salvaneschi. The uphill journey of FaaS in the open-source community. *Journal of Systems and Software*, 198:111589, 2023.
- [44] Fastly. Compute@edge. <https://docs.fastly.com/products/compute-at-edge>, 2023.
- [45] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation*, pages 237–247, Albuquerque New Mexico USA, June 1993. ACM.
- [46] Sadjad Fouladi, Riad S. Wahby, Brennan Shacklett, Karthikeyan Vasuki Balasubramaniam, William Zeng, Rahul Bhalaria, Anirudh Sivaraman, George Porter, and Keith Winstein. Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 363–376, Boston, MA, March 2017. USENIX Association.
- [47] Apache Software Foundation. Apache Beam. <https://beam.apache.org/>, 2024.
- [48] Alexander Fuerst and Prateek Sharma. FaasCache: Keeping Serverless Computing Alive with Greedy-dual Caching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '21*, page 386–400, New York, NY, USA, 2021. Association for Computing Machinery.
- [49] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, Ludmila Cherkasova, and Gabriel Parmer. Sledge: a serverless-first, lightweight wasm runtime for the edge. In *Proceedings of the 21st International Middleware Conference, Middleware '20*, page 265–279, New York, NY, USA, 2020. Association for Computing Machinery.
- [50] Samuel Ginzburg, Mohammad Shahradd, and Michael J. Freedman. VectorVisor: A binary translation scheme for Throughput-Oriented GPU acceleration. In *2023 USENIX Annual Technical Conference (USENIX)*

- ATC 23), pages 1017–1037, Boston, MA, July 2023. USENIX Association.
- [51] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. Enhanced Operating System Security through Efficient and Fine-Grained Address Space Randomization. In *Proceedings of the 21st USENIX Conference on Security Symposium*, Security '12, 2012.
 - [52] Nikolas Roman Herbst, Samuel Kounev, and Ralf H. Reussner. Elasticity in cloud computing: What it is, and what it is not. In Jeffrey O. Kephart, Calton Pu, and Xiaoyun Zhu, editors, *ICAC*, pages 23–27. USENIX Association, 2013.
 - [53] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-parallel Programs From Sequential Building Blocks. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems 2007*, EuroSys '07, page 59–72, New York, NY, USA, 2007. Association for Computing Machinery.
 - [54] Serhii Ivanenko, Carlos Segarra, and Rodrigo Bruno. Mosaic: Optimizing cloud resource efficiency with lazily-packaged application modules. In *Proceedings of the 3rd Workshop on Serverless Systems, Applications and Methodologies*, SESAME' 25, page 21–29, New York, NY, USA, 2025. Association for Computing Machinery.
 - [55] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019.
 - [56] Zhipeng Jia and Emmett Witchel. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, pages 691–707, New York, NY, USA, October 2021. Association for Computing Machinery.
 - [57] Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shravan Narayan, Stefan Savage, Deian Stefan, and Fraser Brown. WaVe: a verifiably secure WebAssembly sandboxing runtime. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, May 2023.
 - [58] Evan Johnson, David Thien, Yousef Alhessi, Shravan Narayan, Fraser Brown, Sorin Lerner, Tyler McMullen, Stefan Savage, and Deian Stefan. Доверяй, но проверяй: SFI safety for native-compiled Wasm. In *NDSS*, 2021.
 - [59] Artjom Joosen, Ahmed Hassan, Martin Asenov, Rajkarn Singh, Luke Darlow, Jianfeng Wang, and Adam Barker. How does it function? characterizing long-term trends in production serverless workloads. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, SoCC '23, page 443–458, New York, NY, USA, 2023. Association for Computing Machinery.
 - [60] Jeongchul Kim and Kyungyong Lee. FunctionBench: A Suite of Workloads for Serverless Cloud Function Service. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, pages 502–504, Milan, Italy, July 2019. IEEE.
 - [61] Ana Klimovic, Yawen Wang, Patrick Stuedi, Animesh Trivedi, Jonas Pfefferle, and Christos Kozyrakis. Pocket: Elastic Ephemeral Storage for Serverless Analytics. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
 - [62] Moiz Kohari. Anomaly Detection from Log Files: The Performance at Scale Use Case. <https://blog.min.io/anomaly-detection-from-log-files-the-performance-at-scale-use-case/>, 2023.
 - [63] Ricardo Koller and Dan Williams. Will Serverless End the Dominance of Linux in the Cloud? In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, HotOS '17, page 169–173, New York, NY, USA, 2017. Association for Computing Machinery.
 - [64] Tom Kuchler, Michael Giardino, Timothy Roscoe, and Ana Klimovic. Function as a Function. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, SoCC '23, 2023.
 - [65] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefevre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the Sixteenth European Conference on Computer Systems*, EuroSys '21, page 376–394, 2021.
 - [66] Collin Lee and John Ousterhout. Granular Computing. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HotOS '19, page 149–154, New York, NY, USA, 2019. Association for Computing Machinery.
 - [67] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 574–587, New York, NY, USA, 2023. Association for Computing Machinery.
 - [68] Shih-Wei Li, John S. Koh, and Jason Nieh. Protecting Cloud Virtual Machines from Hypervisor and Host Operating System Exploits. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1357–1374, Santa Clara, CA, August 2019. USENIX Association.
 - [69] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. RunD: a Lightweight Secure Container Runtime for High-density Deployment and High-concurrency Startup in Serverless Computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 53–68, 2022.
 - [70] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. FaaSFlow: Enable Efficient Workflow Execution for Function-as-a-Service. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, 2022.
 - [71] Zijun Li, Chuha Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. DataFlower: Exploiting the Data-flow Paradigm for Serverless Workflow Orchestration. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '23, page 57–72, New York, NY, USA, 2024. Association for Computing Machinery.
 - [72] James Litton, Anjo Vahldiek-Oberwagner, Eslam Elnikety, Deepak Garg, Bobby Bhattacharjee, and Peter Druschel. Light-Weight Contexts: An OS Abstraction for Safety and Performance. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 49–64, 2016.
 - [73] Jiakai Liu. How tokio schedules tasks: A hard lesson learnt. <https://rustmagazine.org/issue-4/how-tokio-schedule-tasks/>, 2023.
 - [74] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David J. Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. Unikernels: Library operating systems for the cloud. pages 461–472, 2013.
 - [75] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chatterji, and Saurabh Bagchi. SONIC: Application-Aware Data Passing for Chained Serverless Applications. pages 285–301, 2021.
 - [76] Filipe Manco, Costin Lupu, Florian Schmidt, Jose Mendes, Simon Kuenzer, Sumit Sati, Kenichi Yasukata, Costin Raiciu, and Felipe Huici. My VM is Lighter (and Safer) than Your Container. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, 2017.
 - [77] Microsoft. Hyperlight wasm: Fast, secure, and os-free. <https://opsource.microsoft.com/blog/2025/03/26/hyperlight-wasm-fast-secure-and-os-free/>, 2025.
 - [78] Microsoft Azure. Azure Guidance for Secure Isolation. <https://learn.microsoft.com/en-us/azure/azure-government/azure-secure-isolation-guidance>, 2023.
 - [79] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martin Abadi. Naiad: a Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, page 439–455, New York, NY, USA, 2013.

- Association for Computing Machinery.
- [80] Patrick O’Neil, Elizabeth O’Neil, Xuedong Chen, and Stephen Revilak. *The Star Schema Benchmark and Augmented Fact Table Indexing*, page 237–252. Springer-Verlag, Berlin, Heidelberg, 2009.
 - [81] Rishikesh Palve. Ultra-fast serverless functions powered by GraalOS. <https://blogs.oracle.com/cloud-infrastructure/post/ultrafast-serverless-functions-powered-by-graalos>, 2023.
 - [82] Qifan Pu, Shivaram Venkataraman, and Ion Stoica. Shuffling, Fast and Slow: Scalable Analytics on Serverless Infrastructure. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019.
 - [83] Pulumi. Toward a Cloud Native Programming Model. <https://www.pulumi.com/whitepapers/point-of-view/>, 2024.
 - [84] Sheng Qi, Xuanzhe Liu, and Xin Jin. Halfmoon: Log-Optimal Fault-Tolerant Stateful Serverless Computing. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, pages 314–330, New York, NY, USA, October 2023. Association for Computing Machinery.
 - [85] Avi Qumranet, Yaniv Qumranet, Dor Qumranet, Uri Qumranet, and Anthony Liguori. Kvm: The linux virtual machine monitor. *Proceedings Linux Symposium*, 15, 01 2007.
 - [86] L. Richardson, M. Amundsen, and S. Ruby. *RESTful Web APIs: Services for a Changing World*. O’Reilly Media, 2013.
 - [87] Zhenyuan Ruan, Seo Jin Park, Marcos K. Aguilera, Adam Belay, and Malte Schwarzkopf. Nu: Achieving Microsecond-Scale resource fungibility with logical processes. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1409–1427, Boston, MA, April 2023. USENIX Association.
 - [88] Rust contributors. Asynchronous Programming in Rust. <https://rust-lang.github.io/async-book/>.
 - [89] Karla Saur, Tara Mirmira, Konstantinos Karanasos, and Jesús Camacho-Rodríguez. Containerized execution of UDFs: an experimental evaluation. *Proc. VLDB Endow.*, 15(11):3158–3171, July 2022.
 - [90] Divyanshu Saxena, Tao Ji, Arjun Singhvi, Junaid Khalid, and Aditya Akella. Memory Deduplication for Serverless Computing with Medes. In *Proceedings of the Seventeenth European Conference on Computer Systems, EuroSys ’22*, page 714–729, 2022.
 - [91] Malte Schwarzkopf. The Remarkable Utility of Dataflow Computing. <https://www.sigops.org/2020/the-remarkable-utility-of-dataflow-computing/>, 2020.
 - [92] Amazon Web Services. AWS Step Functions. <https://aws.amazon.com/step-functions/>.
 - [93] Amazon Web Services. What is RESTful API? <https://aws.amazon.com/what-is/restful-api/>, 2024.
 - [94] Amazon Web Services. Understanding the Lambda execution environment. <https://docs.aws.amazon.com/lambda/latest/operatorguid/e/execution-environment.html>, 2025.
 - [95] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, 2020.
 - [96] Zhiming Shen, Zhen Sun, Gur-Eyal Sela, Eugene Bagdasaryan, Christina Delimitrou, Robbert Van Renesse, and Hakim Weatherpoon. X-Containers: Breaking Down Barriers to Improve Performance and Isolation of Cloud-Native Containers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS ’19*, 2019.
 - [97] Simon Shillaker and Peter Pietzuch. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020.
 - [98] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. Cloudburst: Stateful Functions-as-a-Service. *Proc. VLDB Endow.*, 13(12), 2020.
 - [99] Boris Köpf Stavros Volos. Preventing side-channels in the cloud. Available at <https://www.microsoft.com/en-us/research/blog/preventing-side-channels-in-the-cloud/>, 2024.
 - [100] Dominic Szablewski. QOI - The Quite OK Image Format for fast, lossless image compression. Available at <https://github.com/phoboslab/qoi>, 2024.
 - [101] Ariel Szekely, Adam Belay, Robert Morris, and M. Frans Kaashoek. Unifying serverless and microservice workloads with SigmaOS. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP ’24*, page 385–402, New York, NY, USA, 2024. Association for Computing Machinery.
 - [102] Yang Tang and Junfeng Yang. Lambdata: Optimizing Serverless Computing by Making Data Intents Explicit. In *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*, pages 294–303, 2020.
 - [103] OpenCV team. OpenCV. Available at <https://opencv.org/>, 2024.
 - [104] Dimitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. Benchmarking, Analysis, and Optimization of Serverless Function Snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
 - [105] Dimitrii Ustiugov, Dohyun Park, Lazar Cvetković, Mihajlo Djokic, Hongyu He, Boris Grot, and Ana Klimovic. Enabling In-Vitro Serverless Systems Research. In *Proceedings of the 4th Workshop on Resource Disaggregation and Serverless, WORDS ’23*, page 1–7, New York, NY, USA, 2023. Association for Computing Machinery.
 - [106] V8. What is V8? <https://v8.dev/>.
 - [107] Kenton Varda. WebAssembly on Cloudflare Workers. <https://blog.cloudflare.com/webassembly-on-cloudflare-workers>, 2018.
 - [108] vHive Ecosystem. vSwarm - Serverless Benchmark Suite. Available at <https://github.com/vhive-serverless/vSwarm>, 2025.
 - [109] Nicholas C. Wanning, Joshua J. Bowden, Kirtankumar Shetty, Ayush Garg, and Kyle C. Hale. Isolating Functions at the Hardware Limit with Virtines. In *Proceedings of the Seventeenth European Conference on Computer Systems, EuroSys ’22*, 2022.
 - [110] Robert N.M. Watson, Jonathan Woodruff, Peter G. Neumann, Simon W. Moore, Jonathan Anderson, David Chisnall, Nirav Dave, Brooks Davis, Khilan Gudka, Ben Laurie, Steven J. Murdoch, Robert Norton, Michael Roe, Stacey Son, and Munraj Vadera. CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization. In *2015 IEEE Symposium on Security and Privacy*, pages 20–37, 2015.
 - [111] Michael Wawrzoniak, Rodrigo Bruno, Ana Klimovic, and Gustavo Alonso. Boxer: Faast ephemeral elasticity for off-the-shelf cloud applications, 2024.
 - [112] Mike Wawrzoniak, Ingo Müller, Rodrigo Fraga Barcelos Paulus Bruno, and Gustavo Alonso. Boxer: Data analytics on network-enabled serverless platforms. In *11th Annual Conference on Innovative Data Systems Research (CIDR 2021)*, 2021.
 - [113] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. TMO: transparent memory offloading in datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS ’22*, page 609–621, New York, NY, USA, 2022. Association for Computing Machinery.
 - [114] Jietao Xiao, Nanzi Yang, Wenbo Shen, Jinku Li, Xin Guo, Zhiqiang Dong, Fei Xie, and Jianfeng Ma. Attacks are forwarded: Breaking the isolation of MicroVM-based containers through operation forwarding. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 7517–7534, Anaheim, CA, August 2023. USENIX Association.
 - [115] Yutian Yan, Tengfei Tu, Lijian Zhao, Yuchen Zhou, and Weihang Wang. Understanding the performance of webassembly applications.

- In *Proceedings of the 21st ACM Internet Measurement Conference*, IMC '21, page 533–549, New York, NY, USA, 2021. Association for Computing Machinery.
- [116] Zachary Yedidia. Lightweight fault isolation: Practical, efficient, and secure software sandboxing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 649–665, New York, NY, USA, 2024. Association for Computing Machinery.
- [117] Minchen Yu, Tingjia Cao, Wei Wang, and Ruichuan Chen. Following the data, not the function: Rethinking function orchestration in serverless computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1489–1504, 2023.
- [118] Yuan Yu, Michael Isard, Dennis Fetterly, Mihai Budiu, Úlfar Erlingsson, Pradeep Kumar Gunda, and Jon Currey. DryadLINQ: A System For General-Purpose Distributed Data-parallel Computing Using a High-Level Language. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 1–14, USA, 2008. USENIX Association.
- [119] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient Distributed Datasets: A Fault-Tolerant Abstraction For In-Memory Cluster Computing. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*, NSDI'12, page 2, USA, 2012. USENIX Association.
- [120] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The Shift from Models to Compound AI Systems. Available at <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, 2024.